Streamlit tests of Backend connectivity

Part 6

Purpose of the Streamlit UI "tests"

- **Install dependencies**
  - Purpose: Ensure Streamlit and the thin HTTP client (requests) are available so the UI runs.
  - File cited: apps/web/requirements.txt.
- **Set environment (optional)**
  - Purpose: Point the UI at your running API base.
  - Var: REACHY_API_BASE → e.g., http://localhost:8081/api/media.
  - Without this, the client defaults to http://localhost:8081/api/media.
- **Run Streamlit**
  - Purpose: Bring up the UI shell (apps/web/main_app.py) to validate basic rendering and page navigation.
- **Navigate to "02 — Label & Promote"**
  - Purpose: Exercise the new read-only list endpoint and verify wiring:
    - The page calls GET /api/videos/list?split=temp.
    - Confirms UI ↔ API connectivity and that the API can see files under /videos/temp.
- **Observe the Simulate Promote behavior**
  - Purpose: Document the current, expected mismatch so it's visible before we change anything:
    - Current API stub (apps/api/routers/media.py) expects old schema: {"schema_version","clip","target","label","correlation_id"}.
    - UI sends scaffold-style: { video_id, dest_split, dry_run, label? }.

        - This intentionally fails for now; the follow-up step will add an adapter to bridge formats.

Are media.py and gateway.py correctly and completely configured?

Short answer: Partially correct for current goals, not complete, and there's one critical mismatch to fix.

- **apps/api/routers/media.py**
    - What's correct
        - Exposes read-only endpoints needed by the UI:
            - GET /api/media base status.
            - GET /api/videos/list scanning absolute /videos/{split} and returning {"items":[], "total":N, ...}.
        - Keeps the existing stub for POST /api/media/promote (old schema) to avoid breaking prior behavior.
        - Logging configured; health endpoint at /media/health.
    - Gaps / Risks
        - **VIDEOS_ROOT hardcoded**: Uses Path("/videos"). If your media are not under /videos, list will be empty. Should be env-configurable (e.g., MEDIA_VIDEOS_ROOT).
        - Promote payload mismatch with the new UI (by design, to be addressed next).
        - Not implementing GET /api/videos/{id}, GET /api/videos/{id}/thumb, POST /api/relabel, POST /api/manifest/rebuild locally. Those exist as proxies in gateway.py, which is fine if your deployment topology uses a gateway.
        - CORS not set at app level; needed if UI is served from another origin (we can do this in main app configuration).
- **apps/api/routers/gateway.py**
    - What's correct
        - Proxies exist for:

        - POST /api/promote → upstream Media Mover
        - POST /api/relabel
        - POST /api/manifest/rebuild
        - GET /api/videos/{video_id} and /api/videos/{video_id}/thumb
    - Enforces X-API-Version: v1 and Idempotency-Key on mutating operations (good per requirements).
    - Has health/ready/metrics endpoints.
  - Gaps / Risks
    - ⚠ **Upstream path mismatch (needs fix)**:
        - Gateway promotes to f"{MEDIA_MOVER_URL}/api/promote".
        - Media Mover exposes POST /api/media/promote.
        - Result: Gateway's promote proxy will not reach the Media Mover route. We must align the upstream path (either change gateway to /api/media/promote or add an alias in Media Mover).
    - MEDIA_MOVER_URL is hardcoded ("http://10.0.4.130:8081"). Should be env-configurable.
    - httpx.AsyncClient() is created at import time and never closed. Better: create in app startup and close in shutdown to avoid resource leaks.
    - No proxy for GET /api/videos/list. Not required if the UI calls Media Mover directly, but if you later decide to front everything via gateway, add a pass-through list endpoint.
    - CORS not configured if the gateway is the API entrypoint for the UI.
- **apps/api/main.py**
  - Currently does not set root_path. If you run Uvicorn with --root-path /api/media, the app will compute paths correctly for reverse proxies, but for strict conformance we can set FastAPI(..., root_path="/api/media") when the Media Mover app is mounted at that base.

Suggested minimal fixes before/with Step 3

- **Fix the gateway → Media Mover promote path**
  - Update gateway.py to call f"{MEDIA_MOVER_URL}/api/media/promote" or add an alias in media.py at /api/promote that forwards to /api/media/promote. This is the one critical functional mismatch.
- **Make VIDEOS_ROOT configurable**
  - In media.py, read MEDIA_VIDEOS_ROOT from env with default /videos. This avoids the empty-list confusion in dev.
- **Lifecycle-manage the AsyncClient**
  - Create httpx.AsyncClient() on startup and close on shutdown in gateway.py.
- Optional now, recommended soon:
  - Add CORS for the UI origin if UI and API are hosted on different ports/origins in dev.
  - Add GET /api/videos/list proxy in gateway.py if you want the UI to hit only the gateway.
  - Set root_path in apps/api/main.py via env (e.g., API_ROOT_PATH=/api/media) to match deployments behind Nginx.

Next step readiness

- Your Streamlit tests are serving their purpose: verify list works and make the promote schema mismatch visible. With the path mismatch noted above fixed and an adapter added (Step 3), the “Simulate Promote” will return a plan (dry-run) instead of erroring.

If you want, I can propose and apply the smallest possible patch set addressing:

- gateway promote path mismatch,
- env-configurable VIDEOS_ROOT,
- and the Step 3 adapter that accepts {video_id, dest_split, dry_run, label?} and transforms it to the existing stub format.

Say “proceed” and I’ll prepare those patches.